

기계학습 기말 과제

CardioCare : 심장질환 예측을 위한 End-to-End Machine Learning System

202221066 컴퓨터공학과 김지우

1. 문제 정의와 사용 목적

CardioCare은 임상 데이터로부터 심장질환 가능성을 예측하여 의료진의 의사결정을 보조하는 end to end 머신러닝 시스템이다. 본 프로젝트의 목적은 단순히 분류 모델을 하나 만드는 것이 아닌, 데이터의 이해와 전처리, 모델 학습, 실험 관리, 테스트, 패키징, 추론, 모니터링, 드리프트 탐지까지 포함하는 전체 머신러닝 라이프 사이클을 구현하는 것이다.

핵심 원칙은 “inform, not decide”로 CardioCare는 환자에게 심장질환이 있는지 최종적으로 진단하는 시스템이 아닌, 심장 전문의가 추가 검토할 수 있도록 위험 가능성을 알려주는 보조 도구이다. 모델의 예측 결과는 임상 판단, 환자 병력, 추가 검사 결과, 의료진의 전문적인 해석과 함께 사용되어야 하며, 단독 의사결정 근거로 사용되면 안된다.

이 프로젝트에서는 UCI Heart Disease 계열 데이터를 사용하였다. 원본 target은 다중 클래스 형태일 수 있으므로, target 값이 0인 경우를 정상, 1 이상인 경우를 heart_disease라고 이진화하였다. 이진 분류 문제로 정의한 이유는 본 시스템의 운영 목적이 정상 또는 심장질환 가능성 있음을 일차적으로 구분하여 의료진에게 경고 신호를 제공하는데 있기 때문이다.

임상적인 관점에서 가장 중요한 오류는 False Negative이다. False Negative는 실제로 심장질환 있는 환자를 정상으로 예측하는 경우이다. 이 오류는 환자가 추가 검사나 치료할 기회를 놓칠 수 있기 때문에 특히 위험하다. 따라서 모델 평가는 단순 accuracy만으로 수행하지 않고, balanced accuracy, precision, recall, F1-score, confusion matrix를 함께 사용하였다. 특히 recall과 confusion matrix를 통해 실제 환자를 얼마나 놓치지 않았는지 확인하였다. 본 프로젝트의 주요 산출물은 다음과 같다.

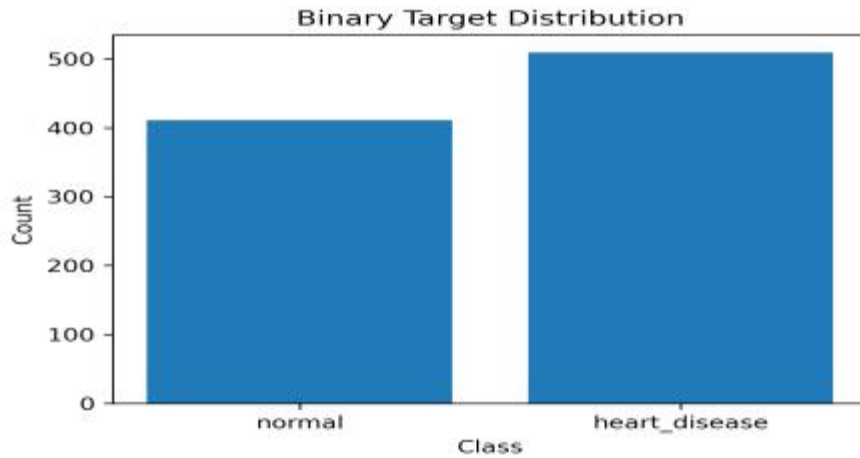
- notebooks/01_eda_preprocessing.ipynb : EDA와 전처리 전략 설계
- src/preprocessing.py : 데이터 로딩, 정제, 전처리 파이프라인 구성
- src/train.py : 모델 학습, 비교, 튜닝, MLflow 기록, 최종 모델 저장
- src/inference.py : 저장된 모델을 이용한 CSV 기반 추론
- src/monitor.py : 데이터 드리프트 시뮬레이션과 성능 변화 분석
- tests/test_pipeline.py : 주요 파이프라인 동작에 대한 unittest
- Dockerfile : 추론 실행을 위한 컨테이너 환경
- .github/workflows/ci.yml : push 또는 pull request시 테스트를 실행하는 CI 워크플로우

2. EDA 핵심 결과

2.1 데이터 구성과 타겟 이진화

본 프로젝트에서 UCI Heart Disease 계열의 여러 원본 파일을 통합하여 사용하였다. 원본 데이터는 age, sex, cp, trestbps, chol, fbs, restecg, thalach, exang, oldpeak, slope, ca, thal, target 등의 변수가 포함되어 있다. 여기에 데이터 출처를 추적하기 위해서 source 컬럼을 추가하였다.

원본 target은 0, 1, 2, 3, 4처럼 다중 클래스일 수 있다. 그래서 여기서는 target이 0이면 정상, 1 이상이면 심장 질환 있음으로 변환하였다.

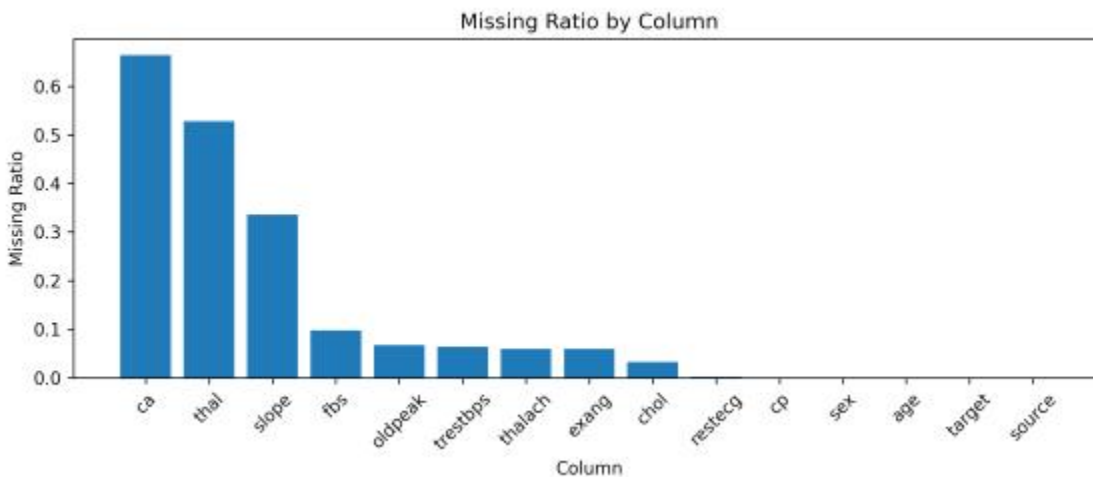


[그림 1. 타겟 클래스 분포]

그림 1은 이진화된 target 분포를 보여준다. 전체 데이터에서 normal은 411개로 약 44.67%이고 heart_disease는 509개로 약 55.3% 임을 알 수 있다. 두 클래스가 극단적으로 불균형하지는 않지만 완전히 균형적이지도 않다. 따라서 단순 accuracy만 사용할 경우 다수 클래스에 유리한 평가가 될 수 있다. 이 때문에 모델 평가는 balanced accuracy, precision, recall, F1-score를 함께 사용하는 것이 좋다고 판단하여 이들을 사용하였다.

2.2 결측치 분포

EDA 결과, 일부 변수에서 결측치 비율이 높게 나타났다. 다음은 컬럼별 결측치 비율을 나타낸 그림이다.



[그림 2. 컬럼별 결측치 비율]

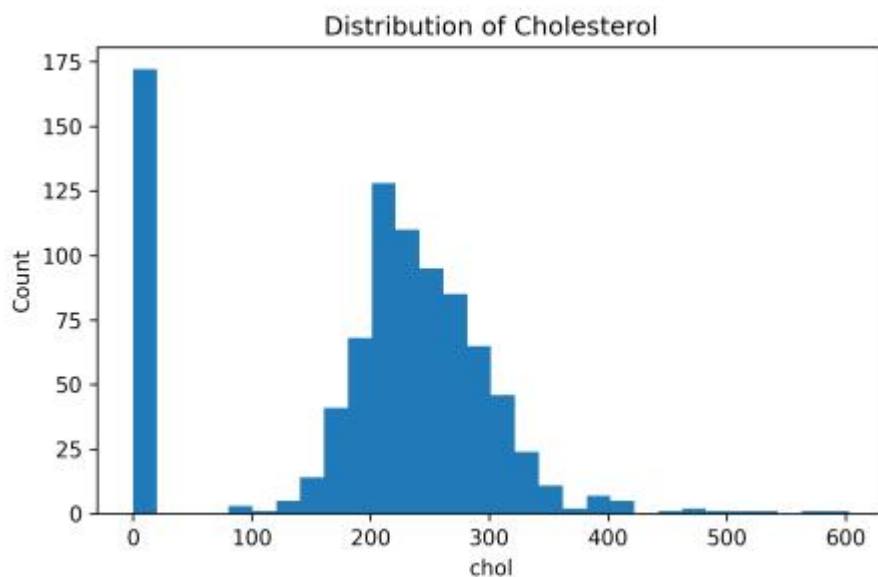
주요 결측치 비율은 다음과 같다.

컬럼	결측치 비율
ca	약 0.6641
thal	약 0.5283
slope	약 0.3359
fbs	약 0.0978
oldpeak	약 0.0674
trestbps	약 0.0641
thalach	약 0.0598
chol	약 0.0326

ca와 thal은 결측치 비율이 각각 약 66.41%, 52.83%로 매우 높았다 결측치가 과도하게 많은 변수는 단순 대체만으로 신뢰할 수 있는 정보를 복원하기 어렵다. 특히 의료 데이터에서는 결측 자체가 병원, 검사 방식, 데이터 수집 시점의 차이를 반영할 수 있기 때문에 무조건 대체하는 것은 위험하다. 따라서 결측 비율이 높은 컬럼은 학습 feature에서 제외하도록 하였다.

반면 chol, thalach, trestbps, oldpeak처럼 결측치 비율이 상대적으로 낮은 연속형 변수는 제거하지 않고 중앙값으로 채워넣었다. 중앙값 대체를 선택한 이유는 의료 수치형 변수에 이상치가 존재할 수 있고, 평균은 이상치의 영향을 더 크게 받기 때문이다.

2.3 연속형 변수 분포와 이상치



[그림 3. 콜레스테롤 분포]

그림 3은 chol 변수의 분포를 보여준다. 콜레스테롤 변수에는 0 근처에 비정상적으로 많은 값들이 존재하며, 높은 값 쪽으로도 일부 극단값이 나타나는 것을 볼 수 있다. 임상적으로 콜레스테롤이 0이라는 값은 실제 측정값이라기보다 결측 또는 기록의 오류로 해석하는 것이 타당하다. 따라서 여기서는 $chol \leq 0$ 과 같은 임상적으로 불가능한 값들을 결측치로 변환한 뒤 전처리 파이프라인에서 대체하도록 처리하였다.

동일한 원칙을 age, trestbps, thalach, oldpeak에도 적용하였다. 예를 들어 나이가 0 이하이거나, 안정시 혈압이 0 이하이거나, 최대 심박수가 0이하인 경우는 실제 임상값으로 보기 어렵다. 이러한 값들은 모델에 그대로 입력할 경우 잘못된 패턴을 학습 시킬 수 있어 NaN으로 변환하였다.

3. EDA 결과에 근거한 전처리 결정

EDA 결과를 바탕으로 전처리 파이프라인을 설계해 보았다. 전처리의 핵심 목표는 다음과 같이 세 가지로 볼 수 있다. 첫 번째는 임상적으로 불가능한 값을 그대로 학습하지 않도록 정제하는 것이고 두 번째는 결측치와 범주형 변수를 새 데이터에도 동일하게 처리할 수 있도록 재사용 가능한 파이프라인을 구성하는 것이다. 세 번째는 데이터 누수를 방지하기 위해 학습 데이터에 대해서만 결측값을 어떤 값을 채워 넣는 작업이나 scaler, encoder, feature selector를 fit한다.

3.1 기본 정제

src/preprocessing.py에서는 여러 원본 파일을 읽어 하나의 DataFrame으로 통합하였다. 이후 다음 정제 과정을 수행하였다.

drop_duplicates()를 사용하여 중복 행 제거를 함으로써 동일 환자 또는 동일 레코드가 반복되어 모델 성능이 과대 평가되는 것을 방지하였다. 그리고 dropna(axis=1, how="all")를 사용하여 완전히 비어있는 컬럼을 제거하였다. 또한, target > 0 이면 1을 사용하여 target 이진화를 수행하여 정상/심장 질환 여부를 구분하는 이진 분류 문제로 전환하였다. 그리고 0 이하의 음수 값을 NaN으로 처리 함으로써 임상적으로 불가능한 값들을 처리하였고, 학습 feature에서 source를 제거하여 병원 출처 편향을 학습할 가능성을 방지하였다.

3.2 결측치 처리

결측치가 많은 컬럼은 제외하였다. 특히 ca, thal처럼 결측 비율이 높은 변수는 대치 후 사용하더라도 실제 정보보다 대치 규칙의 영향을 더 크게 받을 수 있다고 판단하였기 때문이다. 따라서 결측 비율 기준을 두고 높은 결측률을 가진 변수는 feature에서 제외하였다.

수치형 변수에는 중앙값 대치를 적용하였다. 수치형 임상 변수는 이상치가 존재할 수 있기 때문에 평균보다 중앙값이 안정적이라고 판단하여 중앙값 대치를 하였다. 범주형 변수에는 가장 빈번하게 나온 값들로 대치를 하였다. 그 후 one-hot encoding을 수행하였다.

3.3 스케일링과 인코딩

수치형 변수는 SimpleImputer(strategy="median") → StandardScaler()로 처리하였다. 그리고 범주형 변수는 SimpleImputer(strategy="most_frequent") → OneHotEncoder(handle_unknown="ignore")로 처리하였다. 여기서 OneHotEncoder(handle_unknown="ignore")를 사용한 이유는 추론 시 학습 데이터에 없던 범주가 등장하더라도 에러 없이 처리하기 위해서 사용하였다. 의료 데이터는 병원이나 장비 기록 방식에 따라 범주값이 달라질 수 있으므로 추론 안전성이 중요하다.

3.4 데이터 누수 방지

전처리는 ColumnTransformer와 sklearn.pipeline.Pipeline 안에 포함하였다. 이 구조에서는 train/test split 이후 학습 데이터에 대해서만 imputer, scaler, encoder, feature selection이 fit된다. 테스트 데이터는 학습 데이터에서 fit된 변환기를 통해 transform 된다. 이 방식은 데이터 누수를 줄이기 위해 필수적이다. 만약 전체 데이터에 대해 먼저 imputer나 scaler를 fit한 뒤 train/test로 나누게 된다면, 테스트 데이터 분포 정보가 학습 과정에 간접적으로 들어가게된다. 여기서는 이를 방지하기 위해 전처리와 모델을 하나의 pipeline으로 묶었다.

4. MLflow 실험 기반 모델 비교 및 최종 모델 선택

4.1 학습 설정

모델 학습은 src/train.py에서 수행하였다. 전체 데이터는 train_test_split을 사용하여 학습용 데이터와 테스트용 데이터로 나누었다. Random_STATE는 42, TEST_SIZE는 0.2, stratification은 stratify=y 적용, 교차 검증은 5-fold cross validation, 특성 선택은 RandomForestClassifier 기반 SelectFromModel, 튜닝 방식은 Random Forest에 대해 GridSearchCV를 적용하였다.

여기서 stratify=y를 사용한 이유는 train/test split 이후에도 normal과 heart_disease의 클래스 비율을 최대한 유지하기 위해서이다. 클래스 분포가 위에서 봤듯이 완전히 균형적이지 않으므로, stratification을 적용하지 않으면 테스트셋의 클래스 비율이 우연히 달라져 평가 결과가 불안정해질 수 있다.

모든 후보 모델은 preprocessing -> feature_selection -> model과 같은 pipeline으로 구성하였다. 이 구조는 모델별로 동일한 전처리 및 feature selection 흐름을 적용할 수 있게 하며, 학습과 추론 사이의 전

처리 불일치를 줄인다.

4.2 비교한 모델

TN은 실제 정상 환자를 정상으로 예측한 경우, FP는 실제 정상 환자를 심장질환으로 예측한 경우, FN은 실제 심장질환 환자를 정상으로 예측한 경우, TP는 실제 심장질환 환자를 심장질환으로 예측한 경우를 의미한다.

위 결과를 보면 Tuned Random Forest는 test balanced accuracy가 약 0.7873으로 가장 높고, precision도 약 0.8182로 가장 높다. 따라서 전체적인 클래스 균형 성능과 양성 예측의 신뢰도를 함께 고려했을 때 최종 모델로 선택하기에 가장 안정적인 후보라고 판단하였다. 특히 FP가 18건으로 비교 모델 중 가장 낮아, 정상 환자를 심장질환으로 잘못 예측하는 경우를 상대적으로 줄였다.

다만 CardioCare는 의료 보조 시스템이므로 단순히 전체 성능이 높은 모델을 선택하는 것만으로는 충분하지 않다. 임상적 관점에서는 False Negative, 즉 실제 심장질환 환자를 정상으로 예측하는 오류가 특히 중요하다. 이 오류는 환자가 추가 검사나 치료 기회를 놓칠 수 있다는 점에서 위험하다. 이 기준에서 보면 SVC는 FN이 19건으로 가장 적고 recall도 0.8137로 가장 높다. 반면 최종 모델로 선택한 Tuned Random Forest는 FN이 21건이므로, SVC보다 실제 심장질환 환자 2명을 더 정상으로 예측하였다.

따라서 본 프로젝트에서는 balanced accuracy와 precision, 전반적인 성능 균형을 근거로 Tuned Random Forest를 최종 모델로 선택하되, 실제 운영 전에는 recall을 높이고 FN을 줄이기 위한 추가 검토가 필요하다. 구체적으로는 classification threshold 조정, probability calibration, 또는 recall을 더 강하게 반영한 모델 선택 기준을 적용하여 실제 심장질환 환자를 놓치는 위험을 줄이는 방향으로 개선해야 한다. 이 점은 CardioCare가 inform, not decide 원칙에 따라 의사의 판단을 보조하는 시스템이라는 점에서도 중요하다.

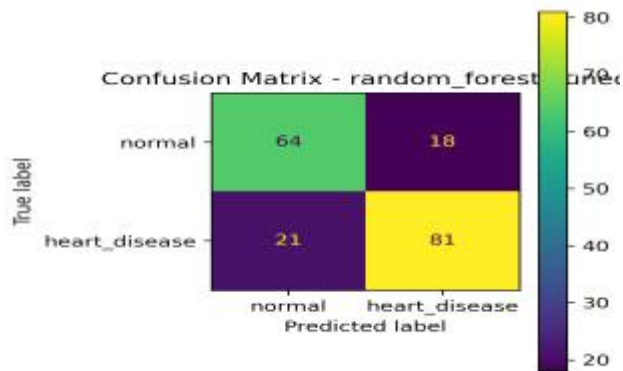
모델명	CV balanced accuracy	Test balanced accuracy	Precision	Recall	F1	TN	FP	FN	TP
SVC	0.7858	0.7727	0.7905	0.8137	0.8019	60	22	19	83
Logistic Regression	0.7801	0.7861	0.8119	0.8039	0.8079	63	19	20	82
Random Forest	0.7635	0.7556	0.7736	0.8039	0.7885	58	24	20	82
Tuned Random Forest	0.7825	0.7873	0.8182	0.7941	0.8060	64	18	21	81

[표 1. 모델 비교 결과 중 일부]

4.3 최종 모델 선택

최종 저장 모델은 random_forest_tuned이며, 저장 위치는 models/final_model.joblib에 있다. 최종 모델로 Tuned Random Forest를 선택한 이유는 성능 지표의 균형 때문이다. 위 표에서 보았듯이 test balanced accuracy가 약 0.7873으로 비교 모델 중 가장 높았고, precision도 약 0.8182로 가장 높았다. 이는 정상 환자를 과도하게 심장 질환으로 판단하는 False Positive를 상대적으로 줄이면서도 전체 클래스 균형 성능을 유지했다는 의미이다. 하지만 recall은 SVC가 약 0.8137로 가장 높고, Tuned Random Forest는 약 0.7941로 약간 낮다. 임상적으로 False Negative가 가장 중요하다는 점을 고려하면 SVC도 강력한 후보라고 볼 수 있다. 실제 confusion matrix를 보면 SVC의 False Negative는 19건이고, Tuned

Random Forest의 False Negative는 21건이다. 즉, Tuned Random Forest는 SVC보다 실제 심장 질환 환자를 2명 더 정상으로 예측했다는 걸 알 수 있다.



[그림 4. 최종 모델 Confusion Matrix]

이 결과를 해석하면, 모델은 실제 심장 질환 환자 102명 중 81명을 올바르게 심장 질환으로 예측했지만, 21명은 정상으로 예측하였다. 이 21건의 False Negative는 본 시스템의 가장 중요한 위험이다. 따라서 최종 모델을 운영할 때는 기본 분류 임계값만 고정해서 사용할 것이 아니라, 추후 threshold 조정 실험을 통해 recall을 높이고 False Negative를 줄이는 방향을 추가로 검토해야 한다. 최종적으로 Tuned Random Forest를 선택한 것은 balanced accuracy, precision, F1 운영 안정성을 종합적으로 고려한 결정이라고 볼 수 있다. 그러나 의료 보조 시스템이라는 맥락에서는 recall 저하가 환자 안전과 직접 연결될 수 있으므로, 실제 배포 전에는 SVC와 Tuned Random Forest를 threshold 조정 및 calibration 관점에서 추가 비교할 필요가 있다.

5. 테스트와 패키징

5.1 단위 테스트

tests/test_pipeline.py에서는 머신러닝 파이프라인의 핵심 동작을 검증하기 위해 unittest 기반 테스트를 작성하였다. 테스트는 단순히 코드가 실행되는지만 보는 것이 아니라, 실제 추론 시스템에서 문제가 될 수 있는 조건을 확인하도록 구성하였다. 테스트 항목으로는 예측 shape 검증, 예측 확률 범위 검증, 임상 입력 값 범위 검증, 결정론적 예측 검증 이렇게 4개의 항목으로 나뉘었다.

첫 번째 테스트는 추론 결과가 입력 데이터와 같은 행 단위로 생성되는지 확인한다. 배치 예측 시스템에서는 입력과 출력의 행 수가 달라지면 환자별 결과 매칭이 깨질 수 있으므로 중요한 검증이다.

두 번째 테스트는 확률 출력의 신뢰성을 확인한다. predict_proba 결과가 0보다 작거나 1보다 크면 확률로 해석할 수 없으며, 각 행의 확률 합이 1이 아니면 클래스 확률로 사용할 수 없다.

세 번째 테스트는 임상적으로 말이 되지 않는 입력을 검출하는지 확인한다. 예를 들어 chol=999는 본 프로젝트에서 설정한 허용 범위 [0, 600]을 벗어나므로 예외를 발생시켜야 한다. 이는 잘못된 입력이 모델에 그대로 들어가 예측 결과를 왜곡하는 것을 방지한다.

네 번째 테스트는 재현성을 확인한다. 동일한 저장 모델과 동일한 입력에 대해 예측 결과가 매번 달라진다면 운영 환경에서 신뢰하기 어렵다. 따라서 고정된 모델과 샘플 입력에 대해 동일한 예측 결과가 반복되는지 확인하였다.

5.2 Docker 패키징

Dockerfile은 python:3.10-slim 기반으로 작성하였고, requirements.txt 설치 후 src, tests, data, models 디렉터리를 컨테이너 내부로 복사하도록 구성하였다. 기본 실행 명령은 src/inference.py를 실행하

며, data/sample_input.csv와 models/final_model.joblib을 사용해 reports/inference_predictions.csv 및 reports/inference.log를 생성한다. 로컬에서 docker build -t cardiocare:1.0 . 명령으로 이미지 빌드를 확인하였고, docker run --rm cardiocare:1.0 실행 시 샘플 입력 5건에 대한 예측 결과와 심장질환 확률이 정상 출력되는 것을 확인하였다.

5.3 CI 워크플로우

.github/workflows/ci.yml은 push 또는 pull request 발생 시 자동으로 테스트를 실행하도록 구성하였다. CI는 Python 3.10 환경을 설정하고, requirements.txt를 설치한 뒤, models/final_model.joblib과 data/sample_input.csv 존재 여부를 확인하고 unittest를 실행한다. 이를 통해 코드 변경 이후 기본 추론 파이프라인이 깨지지 않았는지 자동으로 검증할 수 있다. 실제 GitHub Actions에서 main 브랜치의 CI 워크플로우가 green 상태로 통과한 것을 확인하였다.

6. 드리프트 결과 및 재학습/피드백 루프 계획

6.1 모니터링 설계

src/monitor.py는 저장된 models/final_model.joblib, data/train_split.csv, data/test_split.csv를 사용하여 입력 데이터 드리프트와 성능 변화를 분석한다. 모니터링 로그에는 model_version, dataset_name, input_shape, predictions, 실제 정답이 있는 경우 actual과 같은 정보가 기록된다. 모델 버전은 모델 파일 이름과 수정 시각을 조합하여 생성하였다. 이 방식은 간단하지만, 어떤 모델이 어떤 시점의 데이터에 대해 예측했는지 추적할 수 있게 한다.

6.2 Synthetic Drift 생성

드리프트 실험에서는 테스트셋 복사본에 인위적인 분포 변화를 적용하였다. 변화는 연속형 임상 변수에 대해 다음과 같이 주었다.

변수	적용한 변화
chol	+120
trestbps	+40
thalach	-40
oldpeak	+2

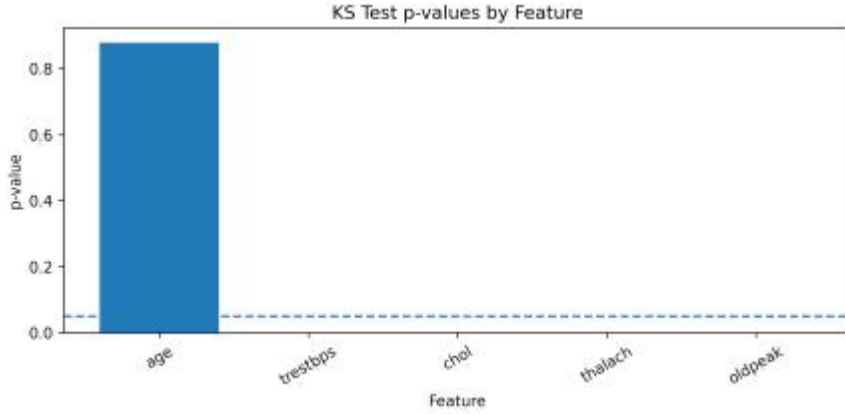
값이 임상적으로 불가능한 범위를 벗어나지 않도록 clipping을 적용하였다. 이 실험은 실제 배포 환경에서 환자군, 측정 장비, 병원 데이터 수집 방식이 변할 때 모델 입력 분포가 달라질 수 있다는 상황을 비슷하게 재현하기 위함이다.

6.3 KS 검정 결과

각 연속형 변수에 대해 학습 분포와 drifted test 분포를 비교하기 위해 Kolmogorov-Smirnov 검정을 사용하였다. 유의수준은 0.05로 설정하였고, p-value가 0.05보다 작으면 드리프트가 발생한 변수로 플래그하였다.

변수	p-value	판정
age	약 0.8793	drift 아님
trestbps	약 6.06e-77	drift 탐지
chol	약 5.19e-76	drift 탐지
thalach	약 4.35e-38	drift 탐지
oldpeak	약 1.70e-87	drift 탐지

[표 2. KS 드리프트 검정 결과 중 일부]



[그림 5. 변수별 KS 검정 p-value]

실험에서 인위적으로 shift를 준 trestbps, chol, thalach, oldpeak는 모두 p-value가 0.05보다 훨씬 작게 나타났다. 반면 shift를 주지 않은 age는 p-value가 약 0.8793으로 드리프트가 감지되지 않았다. 이는 KS 검정이 synthetic drift를 적절히 탐지했음을 보여준다.

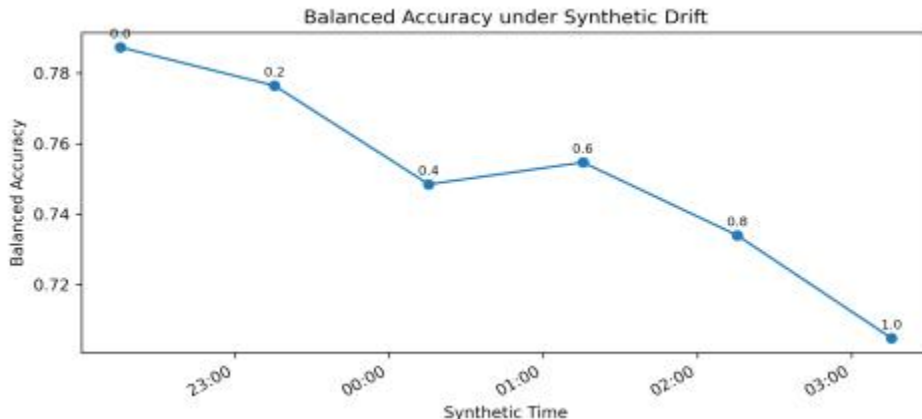
6.4 드리프트에 따른 성능 변화

드리프트가 실제 모델 성능에 어떤 영향을 주는지 확인하기 위해 원본 테스트셋과 drifted 테스트셋의 balanced accuracy를 비교하였다.

데이터셋	샘플 수	Balanced Accuracy	원본 대비 변화량
original_test	184	약 0.7873	0.0000
drifted_test	184	약 0.7048	약 -0.0825

[표 3. 원본 테스트셋과 드리프트 테스트셋 성능 비교 중 일부]

원본 테스트셋에서 balanced accuracy는 약 0.7873이었으나, drifted 테스트셋에서는 약 0.7048로 하락하였다. 변화량은 약 -0.0825이다. 이는 입력 분포 변화가 단순히 통계적으로만 감지된 것이 아니라, 실제 모델 성능 저하와도 연결될 수 있음을 보여준다.



[그림 6. 드리프트 강도 증가에 따른 balanced accuracy 변화]

그림 6은 synthetic drift 강도가 증가함에 따라 balanced accuracy가 전반적으로 감소하는 경향을 보여 준다. 일부 구간에서는 일시적인 반등이 있지만, 최종적으로 드리프트 강도 1.0에서 성능이 가장 낮아진다. 따라서 운영 환경에서는 입력 드리프트와 성능 지표를 함께 모니터링해야 한다. 드리프트가 감지되더라도 성능이 유지될 수 있고, 반대로 작은 분포 변화도 특정 하위 집단에서는 큰 성능 저하를 유발할 수 있기 때문이다.

6.5 재학습 및 피드백 루프 계획

재학습은 자동으로 수행하지 않는다. 의료 도메인에서는 잘못된 데이터나 편향된 라벨로 모델을 자동 재학습하면 기존 편향이 강화되거나 성능이 더 악화될 수 있다. 따라서 핵심 연속형 변수의 KS 검정 p-value가 0.05 미만으로 지속되고, balanced accuracy 또는 recall 하락이 관찰되며, 검증된 정답 라벨이 충분히 수집 될 때 재학습 후보로 올린다.

재학습 승인에는 human-in-the-loop가 필요하다. 의사 또는 데이터 담당자가 라벨 품질, 환자군 변화, 데이터 수집 방식의 변화 여부를 확인한 뒤 재학습 여부를 결정해야 한다.

또한 모델 예측 결과를 검증 없이 다시 학습 데이터에 넣어서는 안 된다. 예측값을 정답처럼 사용하면 모델이 자신의 오류를 다시 학습하는 폭주 피드백 루프가 발생할 수 있다. 따라서 재학습에는 의료진이 검증한 실제 정답 라벨이 있는 데이터만 사용해야 한다.

7. 서빙 선택

여기서는 서빙 방식으로 Model-as-a-Service를 선택한다. Model-as-a-Service는 모델을 병원 내부 서버 또는 중앙 서버에 배포하고, 클라이언트가 API를 통해 예측 요청을 보내는 방식이다.

7.1 지연 시간 관점

심장 질환 위험 예측은 응급 처치처럼 밀리초 단위 응답이 필요한 작업이라기보다, 진료 보조와 추가 검사 판단을 위한 위험도 산출에 가깝다. 따라서 병원 내부망에서 API 형태로 운영하면 허용 가능한 지연 시간 안에서 예측을 제공할 수 있다.

다만 네트워크 상태에 따라 응답 시간이 달라질 수 있으므로, 운영 환경에서는 API 응답 시간, 실패율, 타임아웃 비율을 함께 모니터링해야 한다.

7.2 개인정보 관점

의료 데이터에는 PHI와 같은 민감한 개인정보가 포함될 수 있다. Model-as-a-Service 방식은 입력 데이터가 서버로 전송될 수 있다는 점에서 개인정보 보호 위험이 있다. 이를 줄이기 위해 외부 공개 서버가 아니라 병원 내부망 또는 통제된 보안 환경에서 배포하는 것이 적절하다.

또한 전송 중 암호화, 접근 제어, 인증, 로그 마스킹, 최소 정보 수집 원칙이 필요하다. 추론 로그에는 입력 shape, 예측값, 모델 버전 등 운영에 필요한 정보는 남기되, 환자를 직접 식별할 수 있는 정보는 저장하지 않는 것이 바람직하다.

7.3 업데이트 주기 관점

Model-as-a-Service는 모델 버전 관리, 업데이트, 롤백이 쉽다는 장점이 있다. 모델이 중앙 서버에 배포되어 있으면 여러 클라이언트 장치에 개별적으로 모델을 업데이트할 필요가 없다. MLflow와 연계하면 어떤 모델 버전이 언제 배포되었고, 어떤 지표를 기준으로 선택되었는지 추적할 수 있다.

여기에서는 models/final_model.joblib을 저장 모델로 사용했지만, 실제 운영 단계에서는 MLflow Model

Registry를 사용하여 staging, production, archived 상태를 관리하는 것이 적절하다.

8. 한계, 윤리적 고려 사항, 추가 개선 계획

8.1 한계

첫째, UCI Heart Disease 데이터는 오래된 공개 데이터이며, 실제 최신 병원 환경의 환자군을 완전히 대표하지 못할 수 있다. 환자 인구통계, 진단 기준, 검사 장비, 기록 방식이 현재 의료 환경과 다를 수 있다.

둘째, 여러 원본 파일을 통합했기 때문에 데이터 출처별 편향이 존재할 수 있다. source 컬럼은 학습 feature에서 제외했지만, 다른 임상 변수의 분포 자체가 출처별 차이를 반영할 수 있다. 이 경우 모델은 질병의 일반적 패턴이 아니라 특정 병원 또는 데이터셋의 특성을 학습할 위험이 있다.

셋째, 이 프로젝트에서의 성능 평가는 내부 train/test split과 synthetic drift에 기반한다. 외부 검증 데이터셋에서 동일한 성능이 유지되는지는 추가 확인이 필요하다.

넷째, 최종 모델의 False Negative가 여전히 존재한다. Tuned Random Forest는 테스트셋에서 21건의 False Negative를 보였다. 의료 보조 시스템에서 이 오류는 실제 심장질환 환자를 놓칠 수 있다는 점에서 중요하다.

8.2 윤리적 고려 사항

CardioCare는 “inform, not decide” 원칙에 따라 사용되어야 한다. 모델은 환자의 진단을 단독으로 결정하지 않으며, 의료진에게 참고 정보를 제공하는 보조 도구이다. 예측 결과가 높게 나오더라도 그것이 곧 확정 진단을 의미하지 않고, 낮게 나오더라도 심장질환 가능성이 완전히 배제되는 것은 아니다.

특히 False Negative는 환자 안전에 직접적인 영향을 줄 수 있다. 실제 심장질환 환자를 정상으로 예측하면 추가 검사나 치료가 지연될 수 있다. 따라서 운영 단계에서는 recall 중심의 threshold 조정, 고위험 환자에 대한 보수적 경고 정책, 모델 예측의 불확실성 표시가 필요하다.

개인정보 보호도 중요한 윤리적 고려 사항이다. 의료 데이터는 민감한 개인정보를 포함할 수 있으므로, 접근 권한을 제한하고, 로그에 불필요한 개인 식별 정보를 남기지 않아야 한다. 또한 모델의 성능이 특정 성별, 연령, 병원 출처, 환자군에서 다르게 나타나는지 공정성 검토가 필요하다.

8.3 1주가 더 주어진다면 개선할 점

추가 시간이 주어진다면 첫 번째로 외부 검증 데이터셋을 사용하여 일반화 성능을 확인할 것이다. 현재 결과는 통합 데이터 내부 split에 기반하므로, 실제 다른 병원 또는 최신 환자군에서도 성능이 유지되는지 확인해야 한다.

두 번째로는 threshold 조정 실험을 수행할 것이다. 현재 최종 모델은 balanced accuracy와 precision이 높지만 SVC보다 recall이 낮다. 임상적으로 False Negative를 줄이는 것이 중요하므로, 분류 임계값을 조정하여 recall을 높이는 실험이 필요하다.

세 번째로는 모델 calibration을 수행한다. 예측 확률이 실제 위험도와 얼마나 잘 일치하는지 확인하고, 필요한 경우 calibration을 적용한다. 의료 보조 시스템에서는 단순 클래스 예측보다 신뢰 가능한 확률 추정이 중요하다.

부록. AI 도구 사용

본 프로젝트에서는 코드 구조 점검, 주석 정리, 보고서 초안 구성, 디버깅 방향 검토에 AI 도구를 보조적으로 사용하였다.